

DP Computer Science: A 3.4.4 Distributed Databases (HL Only)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain why maintaining **data consistency** is crucial in a distributed database
- Describe the role of **ACID** (Atomicity, Consistency, Isolation, Durability) in ensuring reliable processing of transactions in distributed databases
- Describe the following features of distributed databases: **concurrency control**, **data consistency**, **data partitioning**, **data security**, **distribution transparency**, **fault tolerance**, **global query processing**, **location transparency**, **replication**, and **scalability**

Key Vocabulary

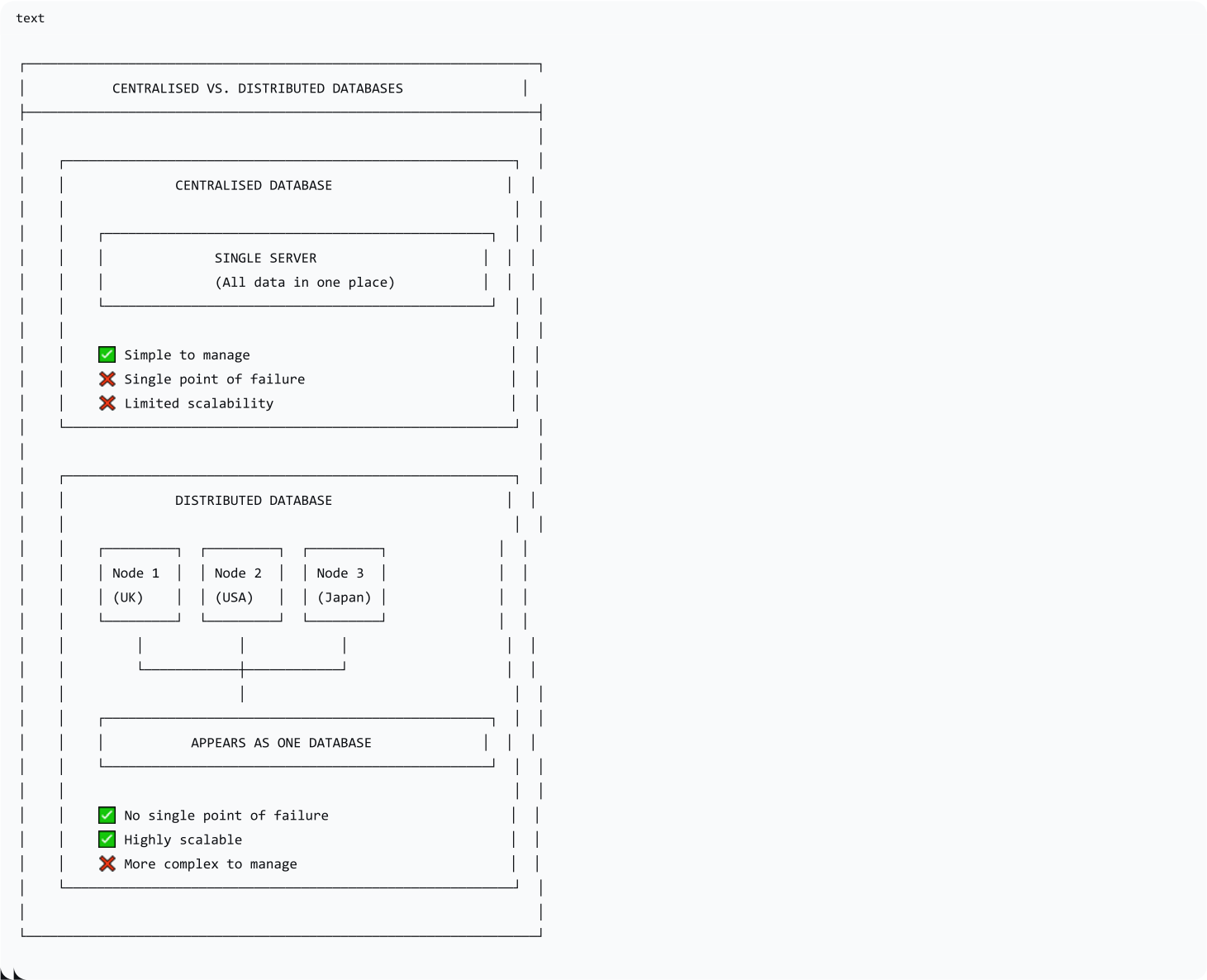
Term	Definition
Distributed database	A database that is stored across multiple interconnected locations (nodes), appearing to users as a single logical database
Centralised database	A database stored in a single location
Data consistency	Ensuring that all copies of data across different nodes are the same
Concurrency control	Managing simultaneous access to data to prevent conflicts
Data partitioning	Splitting a database into smaller pieces (partitions) distributed across nodes
Replication	Creating and maintaining copies of data on multiple nodes
Transparency	The ability to hide distributed complexities from users
Fault tolerance	The ability of a system to continue operating even when some components fail
Scalability	The ability to handle increasing data and user loads by adding more nodes
Global query	A query that accesses data from multiple nodes in a distributed database

Relevant ATL Skills

Skill	Description
Thinking	Critical thinking (analysing challenges and solutions); systems thinking (understanding component interactions); transfer (applying concepts to real-world systems)
Communication	Presenting; explaining complex topics
Self-Management	Organisation; metacognition (reflecting on understanding)

2. What Are Distributed Databases?

Centralised vs. Distributed Databases



Definition

A distributed database is a database that is stored across multiple interconnected locations (nodes), appearing to users as a single logical database.

Why Use a Distributed Database?

Reason	Description
Scalability	Can handle large data volumes by adding more nodes
Fault Tolerance	If one node fails, others continue operating
Performance	Data can be located near users (reduced latency)
Geographic Distribution	Supports global applications with local data access

Reason	Description
Cost	Can use commodity hardware instead of expensive mainframes

Real-World Examples

Application	Distributed Database Example
Global Banking	Customer accounts stored across multiple international data centres
Social Media	User data distributed across servers worldwide (Facebook, Twitter)
E-Commerce	Product inventory distributed across fulfilment centres (Amazon)
Content Delivery Networks (CDN)	Content replicated across global servers (Netflix)
Search Engines	Google's search index distributed across thousands of servers

3. Data Consistency in Distributed Databases

What is Data Consistency?

 Data consistency ensures that all copies of data across different nodes are the same and that transactions leave the database in a valid state.

The Challenge of Consistency

text

THE CONSISTENCY CHALLENGE

Scenario: Two users accessing the same data at the same time across different nodes

User A: UPDATE Balance SET Balance = 200 WHERE ID=1

User B: SELECT Balance FROM Account WHERE ID=1

Which value does User B see?

- The OLD value?
- The NEW value?
- What if they are in different locations?

Without proper consistency mechanisms, users may see different versions of the same data!

Why Consistency is Crucial

Reason	Explanation
Data Integrity	Ensures data is accurate and reliable
User Confidence	Users trust that they see the correct data
Accurate Analysis	Reports and analytics are based on consistent data
Regulatory Compliance	Many regulations require accurate data
Business Operations	Incorrect data can lead to poor decisions

4. ACID Properties in Distributed Databases

What is ACID?

ACID is a set of properties that guarantee reliable processing of database transactions, even in a distributed environment.

The Four Properties



ACID in Distributed Databases

Property	Challenge in Distributed Systems	Solution
Atomicity	Some nodes may commit; others may fail	Two-phase commit protocol
Consistency	Copies on different nodes may diverge	Distributed consensus algorithms
Isolation	Transactions on different nodes may conflict	Distributed locking/transaction management
Durability	A node failure may lose data	Replication; data is written to multiple nodes

5. Features of Distributed Databases

Overview of Features

text

FEATURES OF DISTRIBUTED DATABASES

1. CONCURRENCY CONTROL

2. DATA CONSISTENCY

3. DATA PARTITIONING

4. DATA SECURITY

5. DISTRIBUTION TRANS.

6. FAULT TOLERANCE

7. GLOBAL QUERIES

8. LOCATION TRANS.

9. REPLICATION

10. SCALABILITY

5.1 Concurrency Control

Managing simultaneous access to data to prevent conflicts and ensure data integrity.

Aspect	Description
Purpose	Prevent multiple users from corrupting data when accessing it at the same time
Problem	Two users trying to update the same data simultaneously
Solution	Locking mechanisms, timestamp ordering, optimistic concurrency control
Example	Two customers booking the last seat on a flight—only one should succeed

Techniques:

Technique	Description
Locking	Data is locked when in use; others must wait

Technique	Description
Timestamp Ordering	Transactions are ordered by timestamp to ensure consistency
Optimistic Concurrency	Assume conflicts are rare; check for conflicts before committing

5.2 Data Consistency

| Ensuring all copies of data across different nodes are the same.

Aspect	Description
Purpose	Maintain accuracy and correctness of data
Problem	Different nodes may have different versions of the same data
Solution	Replication protocols, consensus algorithms, two-phase commit
Example	A bank balance must be the same no matter which branch you visit

Consistency Models:

Model	Description
Strong Consistency	All nodes see the same data at all times
Eventual Consistency	All nodes eventually become consistent (used in many NoSQL systems)

5.3 Data Partitioning

| Splitting a database into smaller pieces (partitions) distributed across nodes.

Aspect	Description
Purpose	Distribute data across multiple nodes for scalability
Problem	How to divide data without losing information
Solution	Horizontal partitioning (sharding), vertical partitioning
Example	Customers A-M on Node 1, N-Z on Node 2

Partitioning Methods:

Method	Description	Example
Horizontal Partitioning	Rows are split (by range, hash)	Customers by last name
Vertical Partitioning	Columns are split	Personal data on one node, purchase data on another

5.4 Data Security

| Protecting data from unauthorised access, both within and across nodes.

Aspect	Description
Purpose	Ensure data confidentiality and integrity
Problem	Data is spread across multiple locations; more attack surfaces
Solution	Encryption, access controls, authentication, secure communication
Example	Encrypting data in transit between nodes

5.5 Distribution Transparency

| Hiding the distributed nature of the database from users and applications.

Aspect	Description
Purpose	Make a distributed database appear as a single logical database
Problem	Users shouldn't need to know where data is stored
Solution	A middleware layer that handles location details
Example	Users query the database without knowing which node holds the data

Types of Transparency:

Type	Description
Location Transparency	Users don't need to know the physical location of data
Fragmentation Transparency	Users don't need to know how data is partitioned
Replication Transparency	Users don't need to know which copies exist
Failure Transparency	Users don't see node failures

5.6 Fault Tolerance

| The ability of a system to continue operating even when some components fail.

Aspect	Description
Purpose	Ensure system availability and reliability
Problem	Nodes can fail, networks can disconnect
Solution	Replication, redundancy, failover mechanisms
Example	If one node fails, another node takes over

Fault Tolerance Mechanisms:

Mechanism	Description
Replication	Data is copied to multiple nodes
Failover	If a node fails, traffic is redirected to another

Mechanism	Description
Recovery	Data can be restored from other nodes

5.7 Global Query Processing

| Processing queries that access data from multiple nodes.

Aspect	Description
Purpose	Provide a unified query interface across all data
Problem	Data is spread across nodes; queries must coordinate
Solution	Query decomposition, distributed query optimisation
Example	Joining data from Node 1 and Node 2 in a single query

5.8 Location Transparency

| Users do not need to know the physical location of data they are accessing.

Aspect	Description
Purpose	Hide data location from users
Problem	Data moves; users should not be affected
Solution	A naming scheme that abstracts physical location
Example	Querying for "Customer #1234" without knowing which node holds it

5.9 Replication

| Creating and maintaining copies of data on multiple nodes.

Aspect	Description
Purpose	Improve availability, fault tolerance, and performance
Problem	Maintaining consistency across copies is complex
Solution	Synchronous or asynchronous replication
Example	Same data stored in New York, London, and Tokyo

Replication Strategies:

Strategy	Description
Synchronous	All copies updated before transaction commits (strong consistency)
Asynchronous	Updates sent later; eventual consistency
Master-Slave	One master receives writes; slaves are read-only
Multi-Master	All nodes can accept writes; complex conflict resolution needed

5.10 Scalability

| The ability to handle increasing data and user loads by adding more nodes.

Aspect	Description
Purpose	Support growth without performance degradation
Problem	As nodes increase, coordination becomes harder
Solution	Horizontal scaling (adding more nodes)
Example	Adding a new server to handle more users

Types of Scalability:

Type	Description
Horizontal Scaling	Adding more nodes (scaling out)
Vertical Scaling	Adding more resources to existing nodes (scaling up)

6. Features Summary Table

Feature	Purpose	Key Challenge	Example
Concurrency Control	Prevent simultaneous conflicts	Multiple users updating same data	Transaction locking
Data Consistency	Keep all copies identical	Copies may diverge	Strong consistency, eventual consistency
Partitioning	Distribute data across nodes	How to split data	Sharding by customer ID
Security	Protect distributed data	More attack surfaces	Encryption, access control
Transparency	Hide complexity	Users shouldn't manage distribution	Single logical view
Fault Tolerance	Continue despite failures	Node failures, network issues	Replication, failover
Global Queries	Query across nodes	Coordinating data from multiple sources	Distributed joins
Location Transparency	Hide data location	Data moves, users unaffected	Abstract naming
Replication	Create data copies	Maintaining consistency	Synchronous replication
Scalability	Handle growth	Managing more nodes	Horizontal scaling

7. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Welcome; topic introduction; learning objectives

Phase	Time	Activity
What Are Distributed Databases?	5 min	Define distributed databases; contrast with centralised; real-world examples
The Need for Data Consistency	5 min	Challenges of consistency; problems of inconsistency; importance of unified view
ACID Properties	10 min	Introduction to transactions; explain each ACID property with examples
Features (Part 1)	10 min	Concurrency Control, Data Consistency, Data Partitioning
Features (Part 2)	10 min	Data Security, Distribution Transparency, Fault Tolerance
Features (Part 3)	5 min	Global Queries, Location Transparency, Replication, Scalability
Worksheet Activity	5 min	Students complete matching and scenario exercises
Conclusion	5 min	Summary; Q&A; discussion of real-world applications

Discussion Questions

Question	Purpose
"Why is maintaining consistency more difficult in a distributed database?"	Understand the challenge
"How does replication help with fault tolerance?"	Understand replication benefits
"What is the trade-off between consistency and availability?"	Evaluate trade-offs
"Why is location transparency important for users?"	Understand user perspective

8. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate understanding of features and ACID
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is a distributed database?
2. Why is data consistency important in a distributed database?
3. What is the purpose of data replication?
4. Name two ACID properties and briefly explain each.

9. Differentiation

Level	Strategy
Support	Provide visual aids and analogies; use real-world examples; work in pairs
Challenge	Research specific distributed database technologies (Cassandra, MongoDB, Google Spanner); explore consistency models in detail

10. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.4.4 with examples)
- **Worksheet:** Feature matching and scenario analysis
- **Worksheet Answers**

11. Summary: Key Takeaways

Feature	Key Point
Distributed Database	Stored across multiple locations; appears as one database
Data Consistency	All copies must be the same
ACID	Atomicity, Consistency, Isolation, Durability
Replication	Copies data for availability and fault tolerance
Partitioning	Splits data across nodes
Transparency	Hides complexity from users
Fault Tolerance	Continues even when parts fail
Scalability	Grows by adding more nodes

text

KEY REMINDERS

✔

Distributed = data across multiple locations

✔

Appears as a single logical database

✔

ACID ensures reliable transactions

✔

Consistency ensures all copies match

✔

Replication improves availability

✔

Partitioning distributes data

✔

Transparency hides complexity

✔

Fault tolerance survives failures

✔

Scalability handles growth

"Distributed databases enable global-scale applications by spreading data across multiple locations while maintaining consistency, reliability, and performance through features like replication, transparency, and ACID transactions."